

CSE30301 Homework 2

Implementing Gradient-Based Optimizers

Youngsoo Jang (youngsoo.jang@unist.ac.kr)

Due Date: June 19, 2026, 11:59 PM

Submission: Blackboard

1. Introduction

In previous lectures, we studied gradient descent and several widely used gradient-based optimization algorithms. Gradient descent is one of the most fundamental tools in machine learning, but its behavior can vary significantly depending on the optimization landscape. For example, vanilla gradient descent may oscillate in ill-conditioned problems, move slowly on plateaus, struggle to follow curved valleys, or behave unstably when gradients are noisy. In this assignment, you will implement four standard optimizers and compare their behavior against vanilla gradient descent. The goal is not only to write correct code, but also to understand why different optimizers behave differently across different landscapes. You will use visualization results to debug your implementation, tune hyperparameters, and analyze the final optimization trajectories in your report.

2. Project Instruction

2.1. Optimization Problem

We consider the problem of minimizing a differentiable objective function:

$$\min_{\theta \in \mathbb{R}^d} f(\theta).$$

At each iteration t , an optimizer receives the current parameter vector θ_t and gradient $g_t = \nabla f(\theta_t)$, then computes the next parameter vector θ_{t+1} . The simplest method is vanilla gradient descent:

$$\theta_{t+1} = \theta_t - \eta g_t,$$

where $\eta > 0$ is the learning rate. Vanilla gradient descent is provided in the starter code as a baseline. Your task is to implement and tune more advanced optimizers.

2.2. Advanced Optimizers

This section summarizes the optimizers you must implement. In all equations below, g_t denotes the gradient at iteration t , η denotes the learning rate, and operations such as g_t^2 and $\sqrt{v_t}$ are applied elementwise.

2.2.1 Vanilla Gradient Descent

Vanilla gradient descent updates parameters directly using the current gradient:

$$\theta_{t+1} = \theta_t - \eta g_t.$$

This method is simple and intuitive, but it uses the same learning rate for all dimensions and does not use any information from previous gradients. It is already implemented in the provided code.

2.2.2 Momentum

Momentum accumulates past gradients into a velocity vector. A common update rule is:

$$\begin{aligned}v_t &= \beta v_{t-1} + g_t, \\ \theta_{t+1} &= \theta_t - \eta v_t,\end{aligned}$$

where $\beta \in [0, 1)$ controls how much past gradient information is retained. Momentum can reduce zig-zag behavior by accumulating consistent directions and smoothing oscillatory directions.

2.2.3 RMSProp

RMSProp keeps an exponential moving average of squared gradients:

$$\begin{aligned}s_t &= \beta s_{t-1} + (1 - \beta)g_t^2, \\ \theta_{t+1} &= \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \epsilon},\end{aligned}$$

where ϵ is a small constant for numerical stability. RMSProp adaptively rescales each coordinate by the recent magnitude of its gradients. This can help when different dimensions have different curvature or gradient scales.

2.2.4 Adam

Adam combines momentum-like first moment estimation with RMSProp-like second moment estimation:

$$\begin{aligned}m_t &= \beta_1 m_{t-1} + (1 - \beta_1)g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2)g_t^2.\end{aligned}$$

Because m_t and v_t are initialized at zero, Adam uses bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

The parameter update is:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Adam is often robust because it combines directional smoothing and coordinate-wise adaptive scaling.

2.2.5 AdamW

AdamW modifies Adam by decoupling weight decay from the gradient-based adaptive update. First, compute the Adam direction:

$$d_t = \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Then apply decoupled weight decay:

$$\theta_{t+1} = \theta_t - \eta d_t - \eta \lambda \theta_t,$$

where $\lambda \geq 0$ is the weight decay coefficient. Unlike ordinary L_2 regularization, AdamW does not add $\lambda \theta_t$ to the gradient before adaptive scaling. Instead, it directly shrinks the parameter vector as a separate term.

2.3. Objective of This Assignment

You will implement the following four optimizers inside `student_optimizers.py`:

1. Momentum
2. RMSProp
3. Adam
4. AdamW

For each optimizer, you should correctly implement the update rule and search for effective hyperparameters. You will then run `visualize.py` to generate optimization trajectory GIFs and final-iteration figures. There are no hidden environments in this assignment. All environments used for testing and analysis are provided. Therefore, your main goal is to produce correct implementations, tune the optimizer hyperparameters carefully, and explain the observed behavior clearly.

3. Optimization Environments

This assignment provides four optimization environments. Each environment is designed to highlight a different difficulty of gradient-based optimization.

3.1. Env 1: Zig-Zag

This environment has very different curvature scales in different directions. Vanilla gradient descent can show zig-zag behavior because a learning rate that is stable in the steep direction may be too small in the flat direction.

3.2. Env 2: Plateau

This environment contains regions where the gradient magnitude is very small. Optimizers may move slowly if they cannot adapt to small gradients or accumulate useful directional information.

3.3. Env 3: Rosenbrock

The Rosenbrock function contains a narrow curved valley. This environment tests whether an optimizer can follow a curved path toward the optimum without becoming unstable.

3.4. Env 4: Noisy Saddle Escape

This environment includes saddle-like behavior and noisy gradients. It tests whether an optimizer can handle unstable gradient directions and stochasticity.

4. Code Structure

You are given the following files.

- `optimization_core.py`: contains the core optimization environment, vanilla gradient descent baseline, trajectory recording, and utility functions. Do not modify this file.
- `public_envs.py`: defines the four released optimization environments. Do not modify this file.
- `student_optimizers.py`: contains the skeleton code for Momentum, RMSProp, Adam, and AdamW. This is the main file you must modify and submit.

- `visualize.py`: runs the optimizers and generates trajectory GIFs and final-iteration figures.
- `environment.yml`: provides a recommended conda environment for running the assignment code.

5. Your Programming Task

Your main task is to complete the `step()` function inside each optimizer class in `student_optimizers.py`. The four classes are:

- Momentum
- RMSProp
- Adam
- AdamW

Important: Function Interface Must Not Change

Please keep the provided class names, method names, input arguments, and output format exactly the same as in the starter code. In particular, each optimizer must return the next parameter vector:

```
return x_next
```

Do not rename the classes or modify the interface of `step(self, x, grad, t)`.

Hyperparameter Tuning

In addition to implementing the update equations, you should tune the hyperparameters of each optimizer. You may change the default values inside the `__init__` functions, such as:

- learning rate η ,
- momentum coefficient β ,
- Adam coefficients β_1 and β_2 ,
- numerical stability constant ϵ ,
- AdamW weight decay λ .

Your final submitted code should contain the hyperparameter values that you found to work best. Your report should explain how you selected them.

6. How to Run the Code

After implementing an optimizer, run:

```
python visualize.py
```

The script will generate results under:

```
plots/gifs/
plots/final_figures/
```

The `plots/gifs/` directory contains trajectory animations. The `plots/final_figures/` directory contains high-resolution figures at the final iteration. You should use these outputs to debug your implementations and to prepare your report. If an optimizer has not been implemented yet and still raises `NotImplementedError`, it will be skipped during visualization. This allows you to implement and test one optimizer at a time.

7. Submission Instructions

Please submit your result as one compressed file through Blackboard. Create a single zip file named:

`{student_ID}.zip`

where `student_ID` should be replaced by your actual student ID number. This zip file must contain the following two items.

7.1. Python Source File

Submit your implemented optimizer file. Rename:

`student_optimizers.py`

to:

`student_optimizers_{student_ID}.py`

For example:

`student_optimizers_20251234.py`

This naming rule is important for efficient grading, so please follow it exactly.

7.2. Report

Submit a short report in PDF format. The file name should be:

`report_{student_ID}.pdf`

A recommended length is 2-3 pages. Your report should include:

- a brief explanation of how each optimizer differs from vanilla gradient descent,
- the final hyperparameters you used for each optimizer,
- how you searched for or selected those hyperparameters,
- final-iteration figures from `plots/final_figures/`,
- comparison of optimizer behavior across the four environments,
- a short discussion of which optimizer worked best in each environment and why.

8. Evaluation

Your final grade will be based on the following components.

Implementation Correctness (40%)

Your implementations of Momentum, RMSProp, Adam, and AdamW should correctly follow the mathematical update rules. The code should run without errors and produce reasonable trajectories on the four provided environments.

Hyperparameter Search and Results (20%)

You should tune the hyperparameters of each optimizer and report the final values you selected. The goal is not to obtain the best numerical result. Instead, you should demonstrate meaningful experimentation and justify your choices.

Report and Analysis (40%)

Your report will be graded based on clarity, mathematical correctness, quality of visual evidence, and depth of analysis. A strong report should explain not only which optimizer performed well, but also why it behaved that way in each environment.

9. Use of AI Tools

The use of AI tools is permitted for this assignment. This includes using AI assistants for background research, coding support, debugging help, writing assistance, and organizing your report. However, simply copying AI-generated code or explanations without understanding is strongly discouraged. The core purpose of this assignment is for you to understand how gradient-based optimizers work and to reason about their behavior through experimentation. Your final report should reflect your own implementation, testing, hyperparameter search, and interpretation.

10. Final Note

You are not expected to design a new optimizer from scratch. This assignment is designed for you to implement important standard optimizers, test them on meaningful optimization landscapes, and develop intuition for how optimization algorithms behave in practice. Use the GIFs and final figures actively while debugging and analyzing your results.

A. Appendix: Environment Setup

A.1. Python Environment Setup

If Python is not already installed on your computer, we recommend using Anaconda, which provides Python together with commonly used scientific packages such as NumPy and Matplotlib. Anaconda can be downloaded from:

<https://www.anaconda.com/products/distribution>

After installing Anaconda, open a terminal or Anaconda Prompt and move to the assignment folder.

A.2. Creating a Virtual Environment from YAML File

A YAML file is provided to help you reproduce the recommended Python environment. Suppose the provided file is named:

`environment.yml`

Then create the environment by running:

```
conda env create -f environment.yml
```

This will automatically install the required Python packages.

A.3. Activating the Environment

After creation, activate the environment using:

```
conda activate cse30301
```

A.4. Running the Assignment Code

After activation, run:

```
python visualize.py
```

The generated GIFs and final figures will be saved under:

```
plots/gifs/  
plots/final_figures/
```